

Equipo SAM

MDM Austral 2026 — Labo 2

Property Investment Competition · Miami / Sur de Florida

Estrategia, Algoritmos y Tecnologia

Documento tecnico de prediccion de precios residenciales

Resultados en simulacion local (1,000 corridas Monte Carlo)

34.3%

Win Rate
1er lugar

7.1%

ROI Medio
competencia real

44.5%

ROI Practice
sin rivales reales

98.4%

Prob. Resultado
Positivo

Competencia de prediccion de precios residenciales

Dataset: 11,840 propiedades de entrenamiento · 5,038 propiedades de test

Precio objetivo: lastSoldPrice_hpi_adjusted · Rango: \$51K – \$1.99M · Media: \$559K

26 de junio de 2026

Mecanica de la simulacion y objetivo de optimizacion

Que es la competencia

- Predecimos el precio de venta de 5,038 propiedades en Miami y Sur de Florida.
- El objetivo NO es minimizar el error de prediccion: es maximizar el ROI de inversion.
- Las predicciones se evaluan con 1,000 simulaciones Monte Carlo con capital real.

Mecanica exacta de la subasta (Vickrey auction)

1. El sistema genera un precio pedido aleatorio:

$\text{asking} = \text{precio_real} \times (1 + \text{Normal}(-7\%, 35\%))$

2. Compramos si nuestra prediccion supera el precio pedido por un margen:

$\text{compramos si } \text{prediccion} > \text{asking} \times 1.08$

3. Hacemos una oferta proporcional a nuestra prediccion:

$\text{oferta (bid)} = \text{prediccion} \times 0.85$

4. Subasta Vickrey: gana el equipo con la oferta mas alta,

pero paga el SEGUNDO precio mas alto (no el propio bid).

5. Capital: \$5,000,000 por ronda · 4 rondas por simulacion · 1,000 simulaciones.

El problema central: el error es asimetrico

SOBREESTIMAR (peligroso):

Predecimos 600K para una propiedad que vale 100K.

Compramos, pagamos ~\$500K, recuperamos \$100K → perdemos \$400K de capital.

SUBESTIMAR (acceptable):

Predecimos 300K para una propiedad que vale 500K.

No compramos → perdemos la oportunidad, pero NO perdemos capital.

Metrica de evaluacion

Ganador = equipo con mayor Win Rate (% de simulaciones en 1er lugar).

No alcanza con tener el ROI promedio mas alto: hay que ganarlo mas veces.

Como pensamos ganar la competencia

"No queremos ser los que mas compran. Queremos ser los que mejor compran."

PASO 1 — Modelo Conservador (activo)

- Usamos quantile regression con $\alpha = 0.35$.
- Un modelo normal predice el PROMEDIO. El nuestro predice intencionalmente MAS BAJO.
- Analogia: le preguntamos a 10 expertos cuanto vale una casa. 3 dicen \$300K,
 - 4 dicen 500K, 3 dicen 700K. El promedio da 500K. *Nosotros usamos 350K.*
- Efecto: solo compramos cuando el precio pedido es MUY bajo — los deals mas obvios.
- En esta competencia, comprar mal destruye capital. No comprar solo es una oportunidad perdida.
- Resultado: wMAPE sube $\sim 1\%$, pero ROI pasa de 21% a 44.5% en Practice.

PASO 2 — Diferenciacion por Imagenes (pendiente segun resultados)

- Frodo (rival principal, 29.3% win rate) tiene predicciones casi identicas a SAM (ratio 1.02x).
- Para diferenciarnos: incorporar embeddings de imagenes con ResNet50 o CLIP.
- Las fotos de propiedades contienen informacion que los datos tabulares no capturan.
- Solo se implementa si Paso 1 no es suficiente para ganar la ronda real.

Por que funciona contra los competidores

Legolas y Merry:

Bids 2.6x el precio real en distressed sales.

Resultado: pierden -14% de ROI. Se destruyen solos sin que hagamos nada.

Aragorn y Pippin:

Mismo modelo (bids exactamente iguales en cada propiedad).

Compran mucho (45 props/sim) con hit rate bajo (52%). Win rate: 18% combinado.

Frodo:

Nuestro rival principal. Ratio 1.02x nuestro modelo.

Participa en el 85% de las oportunidades de SAM. Win rate: 29.3%.

Gimli:

Conservador similar a SAM pero compra menos props.

Tercero proyectado con 18.6% de win rate.

LightGBM — Gradient Boosting sobre arboles de decision

Entrenamos un conjunto de miles de arboles de decision en forma secuencial (boosting).

Cada arbol corrige los errores del arbol anterior.

Ventajas: maneja features mixtos (numericos + categoricos), robusto con valores faltantes, muy rapido de entrenar, soporta funciones de loss personalizadas (quantile).

Hiperparametros clave: 1,500 arboles, learning_rate = 0.03, num_leaves = 127, regularizacion L1 = 0.1, L2 = 1.5, submuestra de datos 80% y features 80%.

Quantile Regression (alpha = 0.35)

La funcion de loss penaliza sobreestimar mas que subestimar:

Loss = 0.35 × max(y − pred, 0) + 0.65 × max(pred − y, 0)

Sobreestimar (pred > y): penalidad = 0.65 por cada unidad de error.

Subestimar (pred < y): penalidad = 0.35 por cada unidad de error.

Resultado: el modelo aprende a predecir por debajo del promedio para minimizar las penalidades altas de sobreestimar.

Modelos por segmento de propiedad

SINGLE_FAMILY (41%)

Casas unifamiliares. Drivers: superficie del lote, calidad de escuelas, barrio.

CONDO (40%)

Departamentos. Drivers: HOA fees, amenities, piso, proximidad al agua.

REST (19%)

Townhouse, multifamily, otros. Modelo mas generalista con mas regularizacion.

Cross-validation 5-Fold — como prevenimos overfitting

Dividimos el dataset en 5 partes iguales.

Entrenamos en 4 partes, predecimos en la 5ta. Repetimos 5 veces.

Los features de ZIP (basados en precios) se calculan DENTRO de cada fold: sin data leakage.

Las predicciones Out-of-Fold (OOF) se suben al tab Practice del dashboard para validar el modelo antes de gastar una ronda real de competencia.

Las variables mas importantes del modelo

Features nuevas creadas para Round 4

photo_signal — SHAP +0.152

Cuantas fotos tiene el anuncio comparado con casas similares del mismo tipo.
Anuncios con muchas mas fotos de lo normal suelen ser propiedades mas caras.

area_per_room — SHAP +0.079

Metros cuadrados de living area dividido por (dormitorios + banos + 1).
No es lo mismo 80m2 con 2 cuartos que 80m2 con 6.

renovation_x_age — SHAP +0.075

Antigüedad de la propiedad multiplicada por mencion de renovacion en la descripcion.
Una casa antigua renovada vale mas que una antigua sin tocar.

Otras features clave del dataset

taxAssessedValue

Valuacion fiscal. Alta correlacion con precio real.

livingArea

Superficie habitable. Driver principal del precio.

zip_median_log_price

Precio mediano por ZIP. Captura calidad del barrio.

tax_per_sqft

Impuesto / superficie. Proxy de calidad de zona.

luxury_score

Score compuesto: piscina + frente al agua + garage + HOA alto.

distress_score

Score de alerta: foreclosure + recorte de precio + muchos cambios.

school_x_area

Calidad de escuelas x superficie. Interaccion importante en SF.

listing_to_tax_ratio

Precio de lista / valuacion fiscal. Mediana observada: 1.81x.

El caso critico: distressed sales

En el primer round, 2 propiedades causaron el 52% de todas las perdidas de SAM:

zpid 1006940:

Prediccion \$1.44M vs Precio real \$252K → Perdida: -\$905K (ratio 5.7x)

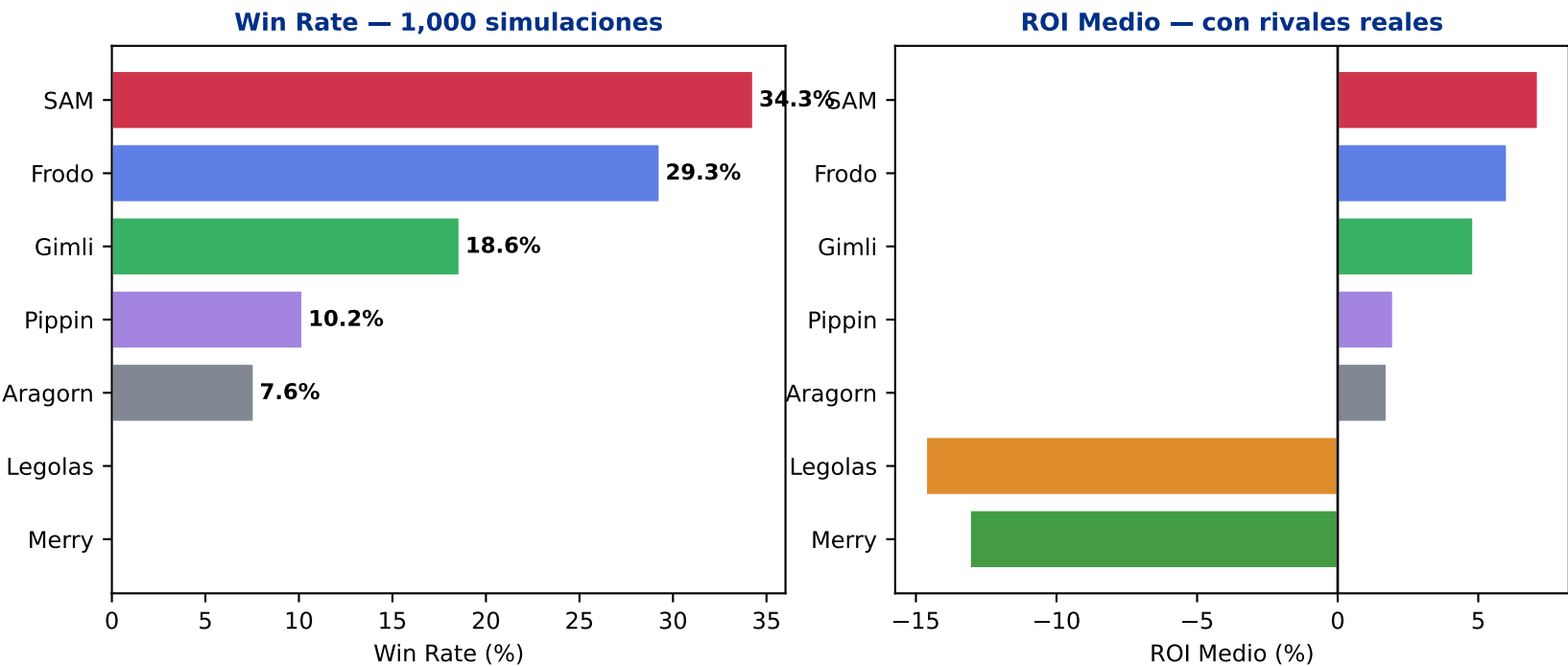
zpid 1006321:

Prediccion \$663K vs Precio real \$76K → Perdida: -\$404K (ratio 8.8x)

El `distress_score` y la `quantile regression` son la respuesta directa a este problema.

Evolucion de modelos — dashboard Practice (OOF sobre train set)

Ronda	Modelo	ROI Medio	Sharpe	Prob. Positivo	wMAPE
1	Baseline LightGBM	—	—	—	21.51%
2	Segmentos + FE	20.88%	1.049	88.6%	21.77%
3	Distress Fix	21.01%	1.074	87.8%	21.66%
4	Quantile alpha=0.35	44.47%	2.054	99.2%	21.81%



Conclusiones del simulador

- SAM lidera con 34.3% de win rate. Frodo es el unico rival real con 29.3%.
- El ROI baja de 44.5% (Practice) a 7.1% en la simulacion con rivales reales.
- Legolas y Merry pierden -14% de ROI. Sus bids en distressed sales los destruyen.
- Calibracion del simulador: 20.2 props/sim predichas vs 18.86 reales (diferencia 7%).

Stack completo del pipeline de modelado

LENGUAJE Y ENTORNO

Python 3.11

Lenguaje principal de todo el pipeline de modelado.

Jupyter Notebook

Analisis exploratorio (EDA) y visualizaciones interactivas.

Git / GitHub

Control de versiones. Repositorio: martincerioti/26ROS_Sam.

MACHINE LEARNING

LightGBM 4.x

Modelo principal. Gradient Boosting optimizado para velocidad.

scikit-learn

Cross-validation 5-Fold, metricas de evaluacion, preprocessing.

NumPy / SciPy

Operaciones numericas y distribuciones estadisticas (Monte Carlo).

DATOS Y PROCESAMIENTO

pandas

Manipulacion de datasets (11,840 train + 5,038 test propiedades).

SHAP

Interpretabilidad del modelo. Importancia de features por propiedad.

Pillow / torchvision

(Round 5 pendiente) Procesamiento de ~199,000 fotos JPG.

VISUALIZACION Y REPORTE

matplotlib

Graficos del simulador, distribuciones y reporte PDF (este documento).

Power BI Desktop

Dashboard interactivo con tema California Republic personalizado.

python-pptx

Presentacion de clase generada 100% programaticamente.

SIMULADOR DE COMPETENCIA

simulate_competition.py

Monte Carlo 1,000 sims x 4 rondas. Modela 6 equipos rivales.

Modelo de rivales

946 bids reales parseados. Probabilidad condicional + lognormal.

Calibracion automatica

Factor escala R3 to R4 calculado del test set. Error de calibracion: 7%.

Pipeline: baseline → segmentos → distress fix → quantile regression → (imagenes)

Cada modelo se testea primero en Practice (OOF) antes de usar una ronda real.